SECURITY REFERENCE

Dorking Encyclopedia

Search Engine Dorking Techniques for Security Testing, OSINT, and Defensive Hardening

Comprehensive Guide to Advanced Search Operators

Google | Bing | DuckDuckGo | Shodan | Censys | FOFA | GitHub

LEGAL WARNING

Every technique in this document is for EDUCATIONAL PURPOSES ONLY.

Use these only on systems you **OWN** or have **WRITTEN PERMISSION** to test.

Using these on systems without permission is a **CRIMINAL OFFENSE** in most countries.

USA

Computer Fraud and Abuse Act (CFAA) - up to 10 years in prison

UK

Computer Misuse Act - up to 10 years in prison

EU

Directive 2013/40/EU - criminal prosecution

Bug Bounty

Only test targets listed in your program scope

The authors of this document are not responsible for how you use it.

Table of Contents

Part I: Google Dorking

1.1 Basic Operators

1.2 Advanced Operators

1.3 File Discovery

1.4 Login Portals

1.5 Exposed Databases

1.6 Config and Secret Files

1.7 Sensitive Documents

1.8 Cameras and IoT

1.9 Vulnerability Discovery

1.10 Cloud Storage Exposure

1.11 Email and People Search

1.12 Subdomain Enumeration

1.13 Google Cheatsheet

Part II: Bing Dorking

2.1 Basic Operators

2.2 Unique Operators

2.3 Bing Cheatsheet

Part III: DuckDuckGo Dorking

3.1 Operators

3.2 Bangs for Security

3.3 DuckDuckGo Cheatsheet

Part IV: Specialized Engines

4.1 Censys

4.2 FOFA

Part V: GitHub Dorking

5.1 Code Search Operators

5.2 API Keys and Secrets

5.3 Config File Leaks
5.4 Database Credentials
5.5 GitHub Cheatsheet

Part VI: Defense

6.1 Remove from Google
6.2 Prevent File Indexing
6.3 robots.txt Guide
6.4 What Attackers Look For

Part VII: Playbooks

7.1 Bug Bounty Recon
7.2 Passive Pentest Recon
7.3 Find Leaked Credentials
7.4 Cloud Exposure Hunt
7.5 Brand Monitoring

Part I: Google Dorking

Google is the most widely used web index, crawling billions of pages and storing snippets and metadata. Knowing the basic operators lets you narrow results to a domain, file type, or phrase — the foundation of effective dorking.

Analogy: imagine Google as a library card catalog where operators let you filter by author (site:), file type (filetype:), title (intitle:), or the exact sentence ("phrase").

1.1 Basic Operators

site: — Limit to Domain

What it does: Limits results to a single domain or host.

Real-world use: Testing a company's public-facing domain and want only results from that site.

```
site:example.com
site:sub.example.com
site:*.example.com
```

Real Examples:

```
site:linkedin.com "works at" "Apple Inc"
site:github.com "AKIA"
site:tesla.com filetype:pdf
```

RISK: LOW

> **Defense**
>
> Use robots.txt to request non-indexing for sensitive paths, avoid publishing secrets, and ensure private subdomains are protected by authentication.

inurl: — Term in URL

What it does: Finds pages with a term in the URL.

Real-world use: Searching for admin panels, staging environments, or phpMyAdmin instances.

```
inurl:admin
inurl:wp-admin
inurl:phpmyadmin
```

Real Examples:

```
site:microsoft.com inurl:admin intitle:login
inurl:phpmyadmin intitle:"phpMyAdmin"
```

RISK: MEDIUM

> **Defense**
>
> Move admin panels behind VPN or IP restrictions; use non-standard URLs and strong multi-factor authentication.

intitle: — Term in Page Title

What it does: Searches for terms in the page title.

```
intitle:"login"
intitle:"index of"
```

Real Examples:

```
intitle:"index of" "parent directory" site:edu
intitle:"Jenkins" "Dashboard"
```

RISK: MEDIUM

intext: — Term in Page Body

What it does: Searches for terms in the page body text.

Use case: Find error messages, API keys mentioned in pages, or copy-paste developer notes.

```
intext:"mysql_connect()"
intext:"DB_PASSWORD"
```

Real Examples:

```
intext:"mysql_connect()" intext:"on line" site:.php
intext:"DB_PASSWORD" site:example.com
```

RISK: MEDIUM

filetype: — Filter by File Type

What it does: Filters results by file extension (pdf, env, sql, xls, etc.).

Use case: Finding spreadsheets or configuration files with sensitive content.

```
filetype:pdf
filetype:env
filetype:sql
```

Real Examples:

```
site:*.gov filetype:xls "social security"
site:github.com filetype:env "DB_PASSWORD"
```

RISK: HIGH

> **Defense**
>
> Remove sensitive files from public web roots, use access controls on file shares, and scan for filetypes in CI.

cache: — View Cached Page

What it does: Shows Google's cached snapshot of a page.

Use case: Viewing a page that has since been removed or investigating deleted content.

```
cache:example.com/some/page
```

RISK: LOW

related: — Find Similar Pages

What it does: Finds pages similar to a given domain.

```
related:github.com
```

Excluding and Combining Operators

Operator	Function	Example

`-term`	Exclude term	`site:example.com -www`
`OR`	Logical OR	`site:example.com (inurl:dev OR inurl:staging)`
`*`	Wildcard	`site:*.example.com`
`..`	Range	`filetype:pdf 2018..2021`
`( )`	Grouping	`(inurl:admin OR inurl:login)`

Quick Reference Table

Operator	What It Does	Example	Risk
`site:`	Limit to one domain	`site:tesla.com`	LOW
`filetype:`	Find filetypes	`filetype:sql`	HIGH
`inurl:`	Terms in URL	`inurl:admin`	MED
`intitle:`	Terms in title	`intitle:"index of"`	MED
`intext:`	Terms in body	`intext:"DB_PASSWORD"`	MED

1.2 Advanced Operators

Advanced operator chaining is where dorking becomes surgical. By combining three or more operators you can find staging environments, error pages revealing software versions, exposed git repositories, and backup files.

Finding Specific Software Versions

Find pages that contain product names and version strings.

```
intext:"Apache/2.4.41 (Ubuntu)" site:example.com
```

Version strings in error pages can be matched to known CVEs like CVE-2021-44228 (Log4Shell).

RISK: MEDIUM-HIGH

Finding Test/Staging Environments

Locate subdomains and URLs that indicate non-production environments.

```
site:target.com (inurl:staging OR inurl:dev OR inurl:test) -www
```

Staging often has weaker access controls or uses real credentials.

RISK: HIGH

Finding Backup and Archive Files

Find zip, tar, bak, and other archived files that may contain sensitive data.

```
site:target.com (filetype:zip OR filetype:bak OR inurl:backup)
```

Finding Exposed .git Repositories

Detect when a site's .git directory is accessible, which can yield full source history.

```
intitle:"Index of /.git" OR inurl:"/.git/config" site:target.com
```

RISK: HIGH

Finding Error Messages

Search for common error text that includes file paths or stack traces.

```
intext:"Warning: mysql_connect()" filetype:php
intext:"Stack trace" "/var/www/" site:example.com
```

Combining Operators — Examples

```
// Find staging MySQL dump files in government sites
site:*.gov inurl:staging filetype:sql

// Find exposed .env files on subdomains
site:target.com inurl:/.env OR filetype:env

// Find public .git folders that include credentials
intitle:"Index of" ".git" "config" site:target.com
```

1.3 File Discovery

Files like .env, .sql, .bak, and .pem can contain secrets and configuration. Developers sometimes push these to web roots or commit them to repositories.

.env Files

Why dangerous: Environment variables storing DB credentials, API keys, and third-party secrets.

```
// Example .env contents:
DB_HOST=prod-db.internal
DB_USER=admin
DB_PASSWORD=SuperSecret123
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
STRIPE_SECRET_KEY=sk_live_abc123xyz
```

Dorks:

```
filetype:env DB_PASSWORD
site:target.com filetype:env
intitle:"index of" ".env"
```

RISK: HIGH

.sql Files

Why dangerous: SQL dumps often contain full database contents, including PII and credentials.

```
filetype:sql "CREATE TABLE"
site:target.com filetype:sql
filetype:sql "INSERT INTO" "users"
```

RISK: HIGH

.bak / Backup Files

```
intitle:"index of" "backup" filetype:bak
```

.log Files

```
filetype:log "password"
site:target.com filetype:log
```

RISK: HIGH

.pem / Private Keys

```
filetype:pem "PRIVATE KEY"
filetype:key "BEGIN RSA PRIVATE KEY"
```

RISK: CRITICAL

Office Files

```
site:*.gov filetype:xls "social security"
site:company.com filetype:pdf "confidential"
```

File Type Risk Summary

File Type	Typical Content	Risk Level
`.env`	API keys, DB credentials	HIGH
`.sql`	Database dumps	HIGH
`.pem/.key`	Private keys	CRITICAL
`.log`	Error messages, tokens	HIGH
`.json/.yaml`	Config, secrets	HIGH
`.htpasswd`	Password hashes	HIGH

1.4 Login Portals

Login portals are high-value targets because they gate access. Finding unadvertised or misconfigured login pages is a staple of reconnaissance.

WordPress Admin

```
inurl:wp-admin intitle:"WordPress"
```

RISK: MEDIUM

cPanel

```
inurl:2082 OR inurl:2083 intitle:"cPanel"
```

RISK: HIGH

phpMyAdmin

```
intitle:"phpMyAdmin" inurl:phpmyadmin
```

RISK: HIGH

Cisco VPN

```
intitle:"Cisco SSL VPN" inurl:webvpn
```

Citrix Gateway

```
intitle:"Citrix Gateway"
```

Outlook Web Access

```
intitle:"Outlook Web App" inurl:owa
```

Developer Tools

```
intitle:"Dashboard [Jenkins]"
intitle:"Grafana"
intitle:"Kibana"
intitle:"Jupyter Notebook" inurl:tree
```

Combining for Admin Panels

```
site:target.com (inurl:admin OR intitle:login) -www -help
```

1.5 Exposed Databases

Databases power most web apps. Exposed database panels, dumps, or admin tools like phpMyAdmin give direct access to data.

MySQL Error Messages

```
intext:"mysql_connect()" intext:"on line" filetype:php
intext:"Warning: mysql_connect()" site:example.com
```

SQL Dump Files

```
filetype:sql "CREATE TABLE"
filetype:sql "INSERT INTO" site:example.com
```

MongoDB / Redis / Elasticsearch

```
intext:"MongoDB" "bind_ip"
intext:"Elasticsearch" "cluster.name" site:example.com
intext:"redis-cli" "Authentication required"
```

Database Connection Strings

```
intext:"mongodb://" site:github.com
intext:"postgres://" filetype:env
```

1.6 Config and Secret Files

Config files and secret material (SSH keys, Docker configs, wp-config.php) are high-value because they directly map to credentials.

Exposed .git Directories

```
intitle:"Index of /.git" OR inurl:"/.git/" site:target.com
```

RISK: HIGH

WordPress wp-config.php

```
filetype:php inurl:wp-config "DB_PASSWORD"
```

SSH Private Keys

```
filetype:pem "PRIVATE KEY" site:github.com
```

RISK: CRITICAL

AWS Credentials

```
filetype:csv "AWSAccessKeyId" OR "AWS_SECRET_ACCESS_KEY"
"AWS_ACCESS_KEY_ID" filename:.env
```

Docker and Kubernetes

```
filename:docker-compose.yml password
filetype:yaml "kind: Pod" "imagePullSecrets"
```

1.7 Sensitive Documents

Documents like PDFs, spreadsheets, and Word files are commonly mis-shared. They may contain PII, financials, or credentials.

PDFs and Docs

```
site:company.com filetype:pdf "confidential"
filetype:doc "internal use only"
```

Spreadsheets

```
site:*.gov filetype:xls "social security"
filetype:csv "employee" "ssn"
```

Text Files and Logs

```
filetype:txt "password"
filetype:log "Authorization: Bearer"
```

1.8 Cameras and IoT

IoT devices and cameras often expose web interfaces or default credentials. Search engines can index their web UIs.

IP Cameras

```
inurl:"/view.shtml" "DVR" OR "IP Camera" site:example.com
inurl:"/snapshot.cgi"
```

Routers and NAS

```
intitle:"D-Link" "router" inurl:login
intitle:"QNAP" "login"
```

1.9 Vulnerability Discovery

Search engines can surface error messages, stack traces, and version strings that map to known CVEs.

Error Messages and Stack Traces

```
intext:"Stack trace" "Exception in thread" site:example.com
intext:"Fatal error" "on line" filetype:php
```

Version Banners

```
intext:"Apache/2.4.41 (Ubuntu)" site:example.com
```

Changelogs and Upgrade Pages

```
site:example.com "CHANGELOG" "v2."
```

1.10 Cloud Storage Exposure

Cloud storage (S3, Google Cloud Storage, Azure blob) often shows up in code, docs, or public links.

S3 Buckets

```
site:amazonaws.com "s3.amazonaws.com" "confidential"
inurl:s3.amazonaws.com "backup"
```

Google Cloud Storage

```
inurl:storage.googleapis.com "project-id"
```

Azure Blob Storage

```
inurl:blob.core.windows.net "container"
```

1.11 Email and People Search

Finding email addresses and people-related info helps map an organization.

Finding Emails

```
"@company.com" site:linkedin.com
site:company.com "@company.com" filetype:pdf
```

Searching for People and Roles

```
site:company.com "works at" "Google"
site:linkedin.com "Chief Technology Officer" "Tesla"
```

Support Emails

```
site:company.com "support@" OR "security@"
```

1.12 Subdomain Enumeration

Subdomains often host staging, admin, API, or developer tools.

Basic Subdomain Discovery

```
site:*.example.com -www
```

Find Interesting Subdomains

```
site:*.example.com inurl:api -www
```

Advanced Tips

- Search for `site:example.com -www -blog -help` to remove noise
- Use `site:example.com "Application Name"` to find related subdomains
- Cross-check with Certificate Transparency logs (crt.sh)

Workflow

1. `site:*.target.com -www -help -support`
2. Save unique subdomains
3. Cross-reference with Shodan for matching services

1.13 Google Dorking Cheatsheet

Quick Operator Reference

Operator	Function
`site:domain`	Limit to a domain
`filetype:ext`	Find documents and files
`inurl:term`	Term in URL
`intitle:term`	Term in page title
`intext:term`	Term in page body
`-term`	Exclude term
`OR`	Logical OR
`"exact phrase"`	Exact match
`cache:`	View cached page

Fast Examples

```
site:tesla.com filetype:pdf
site:github.com "AKIA" filetype:env
intitle:"index of" "backup"
inurl:phpmyadmin intitle:"phpMyAdmin"
filetype:pem "PRIVATE KEY"
```

Risk Levels Quick Key

Level	Description
LOW	Informational - low sensitivity
MEDIUM	Reveals structure or technology
HIGH	Direct sensitive data exposure
CRITICAL	Private keys / full access

Part II: Bing Dorking

Bing is Microsoft's search engine. It supports many of the same operators as Google but also has unique behaviors and filters. Bug hunters use Bing because some sites are indexed differently than Google, and Bing can return different results for the same query.

2.1 Basic Operators

Bing supports many of the same operators as Google (`site:`, `filetype:`, `inurl:`, `intitle:`). Use it as a secondary index to find results that Google might not surface.

Supported Operators

Operator	Function	Example
`site:`	Limit to a domain	`site:example.com`
`filetype:`	Filter by file type	`filetype:env`
`inurl:`	Term in URL	`inurl:admin`
`intitle:`	Term in page title	`intitle:login`
`ip:`	Search by IP address	`ip:93.184.216.34`

Examples

```
site:example.com filetype:env
site:github.com "AKIA"
inurl:admin intitle:login site:example.com
```

RISK: MEDIUM-HIGH depending on query results

2.2 Unique Operators and Tips

Bing offers `ip:` searches and sometimes different indexing behavior.

IP-Based Searches

Use `ip:` to discover pages hosted on the same address:

```
ip:93.184.216.34
ip:93.184.216.34 site:example.com
```

Unique Bing Tips

- Use `ip:` to find webpages hosted on a specific IP
- Bing sometimes indexes dynamically generated pages that Google doesn't
- Bing's webmaster tools can show crawl errors and index status for domains you own

RISK: LOW-MEDIUM

2.3 Bing Cheatsheet

```
site:target.com filetype:env
ip:93.184.216.34
site:github.com "AKIA"
```

Part III: DuckDuckGo Dorking

DuckDuckGo is a privacy-focused search engine. It supports a subset of operators and has a feature called "bangs" that can redirect queries to other sites. Security researchers sometimes use DuckDuckGo because it returns different results and respects fewer personalization signals.

3.1 Operators

DuckDuckGo uses simpler operators but is useful for cross-checks when Google returns sparse or personalized results.

site: — Limit to Domain

```
site:github.com "AKIA"
site:tesla.com filetype:pdf
```

RISK: LOW

filetype: — Filter by Extension

```
filetype:env "DB_PASSWORD"
```

RISK: HIGH

Quoted Phrases and inurl:

Works as expected. Use quotes for exact matches and `inurl:` to find terms in URLs.

3.2 Bangs for Security

DuckDuckGo "bangs" (`!`) are shortcuts that forward your query to other sites.

Useful Bangs for Security

Bang	Function	Example
`!gh`	GitHub search	`!gh AKIA site:github.com`

`!sh`	Shodan (if supported)	Quick link to Shodan searches
`!so`	StackOverflow	Find error messages and fixes

Examples

```
!gh site:github.com "AWS_ACCESS_KEY_ID"
!so "TypeError: undefined is not a function"
```

RISK: LOW — these are convenience features

3.3 DuckDuckGo Cheatsheet

```
site:target.com filetype:env
!gh site:github.com "AKIA"
filetype:pdf "confidential"
```

Part IV: Specialized Engines

Specialized search engines like Censys and FOFA focus on internet-wide scanning and asset discovery. They index services, certificates, and banners rather than web page content.

4.1 Censys

Censys is an internet-wide scanner that focuses heavily on certificates and TLS information. It scans hosts and indexes TLS, SSH, and service metadata.

What Is Censys?

Security teams use Censys to find certificate reuse, expired certs, and hosts by certificate attributes. Use Censys to search by `autonomous_system.asn`, certificate subject, issuer, and subjectAltName.

Basic Queries

```
services.tcp.port:443 and parsed.http.title: "Welcome"
443.https.tls.cipher_suite: "TLS_AES_128_GCM_SHA256"
autonomous_system.asn: 15169
```

Certificate Queries

Certificates reveal hostnames, SANs, and sometimes email addresses and organization names.

```
parsed.extensions.subject_alt_name.dns_names: "example.com"
parsed.subject.common_name: "www.example.com"
parsed.issuer.common_name: "Let's Encrypt"
```

Censys Cheatsheet

```
autonomous_system.asn: 15169
parsed.subject.common_name: "www.example.com"
services.tcp.port: 22 and metadata.os: "Linux"
```

> **Defense**

> Monitor your certificate usage and revoke/reissue reused or expired certs. Audit CT logs for unexpected certificates and set up monitoring for certificates issued for your domains.

4.2 FOFA

FOFA is a Chinese-based internet asset search engine similar to Shodan and Censys. It indexes banners and supports rich query syntax.

Example Operators

Operator	Function	Example
`domain=""`	Search by domain	`domain="example.com"`
`header=""`	Search by header	`header="Server: nginx"`
`body=""`	Search by body content	`body="login"`
`cert=""`	Search by certificate	`cert="CN=example.com"`

Examples

```
header="Server: Apache" and body="index of"
domain="example.com"
cert="CN=example.com"
```

RISK: MEDIUM-HIGH depending on results

FOFA Cheatsheet

```
header="Server: nginx" and body="index of"
domain="example.com"
cert="CN=example.com"
```

Part V: GitHub Dorking

GitHub is a vast index of source code. People accidentally commit secrets and config. GitHub search supports `filename:`, `extension:`, `path:`, and content matching — powerful for finding leaked keys and credentials.

5.1 Code Search Operators

GitHub code search has operators like `filename:`, `path:`, `extension:`, and `repo:`. Use these to focus on likely locations for secrets.

Available Operators

Operator	Function	Example
`filename:`	Find by filename	`filename:.env`
`extension:`	Find by extension	`extension:json`
`path:`	Find by path	`path:.github/workflows`
`repo:`	Limit to repo	`repo:organization/repo`

Real Examples

```
filename:.env "DB_PASSWORD"
extension:yaml "secret"
path:.github/workflows "secrets"
```

RISK: HIGH when secrets are present

> **Defense**
>
> Use GitHub secret scanning, remove secrets from history, and restrict repo visibility.

5.2 API Keys and Secrets

GitHub is a massive index of source code. People accidentally commit secrets: API keys, private keys, tokens, and credentials. These leaks are gold for attackers.

AWS Access Keys

Pattern: `AKIA[0-9A-Z]{16}`

Damage if leaked: Full AWS account misuse — data exfiltration, resource provisioning, crypto-mining.

Dorks:

```
"AKIA" AND ("secret" OR "aws_secret") filename:.env
"AWS_ACCESS_KEY_ID" extension:env
"AWSSecretKey" filename:credentials
path:.aws/credentials
```

Redacted example:

```
AKIAIOSFODNN7EXAMPLE
```

Stripe Keys

Pattern: `sk_live_[A-Za-z0-9]{24}`

Damage: Unauthorized charges, refunds, customer data exposure.

Dorks:

```
"sk_live_" filename:.env
"STRIPE_SECRET_KEY" extension:env
"stripe_secret" language:python
```

Google API Keys

Pattern: `AIza[0-9A-Za-z\-_]{35}`

```
"AIzaSy" filename:.env
"GOOGLE_API_KEY" extension:json
"google_maps_api_key" language:javascript
```

RISK: MEDIUM-HIGH depending on API permissions

SSH Private Keys

An SSH private key is essentially a master key for servers.

```
"BEGIN RSA PRIVATE KEY" extension:pem
"BEGIN OPENSSH PRIVATE KEY" filename:id_rsa
filename:id_ed25519
```

RISK: CRITICAL

Slack / Discord Tokens

Tokens can let an attacker post messages as a bot or access channels.

```
"xoxp-" OR "xoxb-" site:github.com
"discord_bot_token" filename:.env
```

RISK: HIGH

GitHub Tokens (Personal Access Tokens)

Pattern: `ghp_[A-Za-z0-9_]{36}`

```
"ghp_" site:github.com
"GITHUB_TOKEN" filename:.env
```

RISK: CRITICAL

Other Secrets

Service	Pattern
Twilio	`SK[0-9a-fA-F]{32}`
SendGrid	`SG.[A-Za-z0-9_-]{20,}`
Firebase	`AAAA...` (server keys)
Heroku	`[0-9a-f]{32}`

How to Verify a Secret is Real (Non-Destructive)

1. Check repo context (is it a config file for production?)
2. Look for references to production endpoints or AWS ARNs
3. Use public metadata (commit messages, CI YAML) to estimate impact
4. **Never attempt to authenticate with a found key**

Responsible Disclosure Template

1. **Title:** Found exposed secret in public repository
2. **Target:** `https://github.com/org/repo/path`
3. **Evidence:** Screenshot of file with redacted secret
4. **Impact:** Describe what the secret allows
5. **Remediation:** Revoke key, rotate, add secrets scanning, remove from history

5.3 Config File Leaks

Config files often contain credentials. Searching GitHub for config filenames and known keys helps find leaks quickly.

Examples

```
filename:wp-config.php "DB_PASSWORD"
filename:docker-compose.yml "PASSWORD"
filename:.env "AWS_SECRET_ACCESS_KEY"
```

RISK: HIGH

> **Defense**
>
> Remove secrets from history, use secret scanning, and add pre-commit hooks.

5.4 Database Credentials

Database connection strings and credentials in code allow attackers to connect to databases directly.

Dorks and Patterns

```
intext:"postgres://" site:github.com
intext:"mongodb://" filename:.env
"DB_PASSWORD" path:config
```

RISK: HIGH

Defense

Rotate credentials, remove secrets from repos, and use IAM roles instead of embedded usernames/passwords.

5.5 GitHub Dorking Cheatsheet

```
filename:.env "DB_PASSWORD"
"BEGIN RSA PRIVATE KEY" site:github.com
intext:"AKIA" filename:.env
```

Part VI: Defense

This section contains defensive guides: removal from search, preventing file indexing, robots.txt best practices, and attacker checklists.

6.1 How To Remove Yourself From Google

If you find sensitive content indexed by Google, take steps to remove it: remove the content from the web, request removal via Search Console, and ask for cache removal.

Steps

1. **Remove or restrict the resource** — move to private storage
2. **Use Google Search Console** > Removals > Request removal of URL or cached copy
3. **If removal is urgent (PII),** use the legal removals request

> **What's Next**
>
> Prevent re-indexing with `noindex` meta tags or `robots.txt`.

6.2 Prevent File Indexing

Practical steps to prevent files from being indexed:

1. Use robots.txt

Note: Not an access control — well-behaved crawlers only.

```
User-agent: *
Disallow: /private/
```

2. Add noindex Meta Tag

```
<meta name="robots" content="noindex">
```

3. Use HTTP Auth or IP Restrictions

For sensitive directories, require authentication.

4. Move Files to Private Storage

Remove files from public web roots and store in private object storage with ACLs.

6.3 robots.txt Guide

robots.txt tells well-behaved crawlers which paths to avoid. It's not a security control — sensitive files should not rely on it alone.

Example robots.txt

```
User-agent: *
Disallow: /admin/
Disallow: /config/

User-agent: Googlebot
Disallow: /staging/
```

> **Important:** Use `noindex` and access controls in addition to robots.

6.4 What Attackers Look For — Defender's Guide

This section shows what an attacker would search for and how you, as a defender, can check and fix exposure.

Threat: Exposed .env or Config Files

Attacker's goal: Obtain API keys, DB credentials, or tokens.

Dork they'd use:

```
site:yourcompany.com filetype:env OR filetype:yaml "DB_PASSWORD"
```

Fix:

- Remove .env from web root
- Add .env to .gitignore and remove from repo history (use git filter-repo)

- Rotate credentials and store secrets in a manager

Threat: Publicly Accessible Backups and SQL Dumps

Dork:

```
site:yourcompany.com filetype:sql OR intitle:"index of" "backup"
```

Fix:

- Move backups to private storage with proper IAM
- Ensure web servers do not serve backup directories

Threat: Leaked Private Keys

Dork:

```
site:github.com "BEGIN RSA PRIVATE KEY" "yourcompany"
```

Fix:

- Revoke and rotate keys immediately
- Add pre-commit hooks and CI secret scanning

Threat: Exposed Admin Panels

Dork:

```
site:yourcompany.com inurl:admin intitle:login
```

Fix:

- Enforce IP allowlists or place behind VPN
- Require MFA for admin accounts

Threat: Public Cloud Buckets

Dork:

```
site:github.com "s3.amazonaws.com" "yourcompany"
```

Fix:

- Use bucket policies that deny public read
- Monitor bucket ACLs and enable object-level logging

Final Checklist for Defenders

- Run `site:yourcompany.com filetype:env` and review
- Search GitHub for `yourcompany` + `AWS_ACCESS_KEY_ID`
- Ensure directory listing is disabled on all servers
- Add pre-commit secret scanning and CI checks
- Implement least privilege for cloud credentials

Part VII: Playbooks

This section contains mission-style playbooks: step-by-step guides for bug bounty recon, passive pentests, finding leaked credentials, cloud exposure hunts, and brand monitoring.

7.1 Bug Bounty Recon Playbook

The Mission: You've joined a bug bounty program. Scope: `*.shopify.com`. Your mission: perform passive reconnaissance only. No scanning, no brute force — just public information.

Phase 1: Figure Out What They've Built (30 minutes)

Step 1: Map the main domain

```
site:shopify.com
```

Step 2: Find interesting subdomains

```
site:*.shopify.com -www -help -community
```

Step 3: Look for staging/dev environments

```
site:shopify.com inurl:staging OR inurl:dev OR inurl:test
```

What's next? If you find dev/staging, note it for escalation — these often have weaker security.

Phase 2: Find Login Pages and Admin Panels (20 minutes)

Step 1: Generic admin search

```
site:shopify.com inurl:admin intitle:login
```

Step 2: Search specific panels

```
site:shopify.com intitle:"Dashboard [Jenkins]"
site:shopify.com intitle:"Grafana"
site:shopify.com intitle:"Kibana"
```

What you want: Unlinked panels, staging login pages, or pages revealing versions.

Phase 3: Find Code/Config Leaks (30 minutes)

Step 1: Search for config files

```
site:shopify.com filetype:env OR filetype:yaml OR filetype:json
```

Step 2: Search GitHub for org-related secrets

```
site:github.com "shopify" "AWS_ACCESS_KEY_ID"
```

Note: If you find things, do not attempt to use credentials. Instead, prepare a responsible disclosure.

Phase 4: Find Sensitive Documents & Cloud Exposure (30 minutes)

Step 1: Public cloud docs

```
site:drive.google.com shopify
site:amazonaws.com "shopify" "backup"
```

Step 2: S3 references

```
site:github.com "s3.amazonaws.com" "shopify"
```

What you want: Backups, spreadsheets, or links to buckets.

Phase 5: Report & Triage (30 minutes)

1. Prepare evidence: URL, snippet, and screenshot
2. Determine impact and whether it's in-scope
3. Draft a concise report with remediation steps

Quick Live Dork Examples

```
site:shopify.com inurl:staging "index of"
site:shopify.com filetype:sql "INSERT INTO"
site:github.com shopify "AKIA"
```

Tips for the Live Session

- Keep notes and timebox each phase
- Use disposable browser profiles to avoid leaking your own cookies
- If you escalate publicly, redact secrets in screenshots

7.2 Pentest — Passive Recon

This playbook walks through passive reconnaissance for a penetration test: assets, subdomains, dorks, and OSINT aggregation.

Phases

1. **Asset discovery** via search engines and CT logs
2. **Technology fingerprinting** with Shodan/Censys
3. **Public repo and secret search** on GitHub
4. **Document and report** findings

Include real dorks at each step and always avoid active scanning unless permitted.

7.3 Find Leaked Credentials

Step-by-step guide to hunt for leaked credentials across search engines and code hosts. Focus: non-destructive discovery, documentation, and responsible reporting.

Phases

1. **Search GitHub** for `AKIA`, `sk_live_`, `BEGIN RSA PRIVATE KEY` patterns
2. **Use Google dorks** for `filetype:env`, `filetype:sql` and cloud storage links
3. **Monitor paste sites** and specialized engines for new leaks
4. **Triage and report,** recommending rotation and revocation

7.4 Cloud Exposure Hunt

Hunt for cloud misconfigurations: public buckets, leaked IAM keys, and improperly secured services.

Phases

1. **Search for cloud storage references** in code and docs:
   - `s3.amazonaws.com`
   - `storage.googleapis.com`
   - `blob.core.windows.net`
2. **Use Shodan/Censys** to find open management ports for cloud consoles
3. **Check public repos** for long-lived cloud keys and rotate

7.5 Brand Monitoring

Use dorking to monitor brand mentions, leaked credentials, or phishing sites referencing your company.

Phases

1. **Set up persistent Google searches** for `site:` and product names
2. **Monitor GitHub** for references to brand and credentials
3. **Use Pastebin and publicwww** to find copied content

Appendix: Master Quick Reference

Risk Level Legend

Indicator	Level	Description
LOW	Green	Informational — low sensitivity
MEDIUM	Yellow	Reveals structure or technology
HIGH	Red	Direct sensitive data exposure
CRITICAL	Black	Private keys / full system access

Universal Operators Cheat Sheet

Operator	Google	Bing	DuckDuckGo	GitHub
Domain filter	`site:`	`site:`	`site:`	-
File type	`filetype:`	`filetype:`	`filetype:`	`extension:`
URL content	`inurl:`	`inurl:`	`inurl:`	-
Title content	`intitle:`	`intitle:`	-	-
Body content	`intext:`	-	-	`intext:`
Filename	-	-	-	`filename:`
Path	-	-	-	`path:`
Repository	-	-	-	`repo:`
IP search	-	`ip:`	-	-

Common Secret Patterns

Type	Pattern	Example Search
AWS Key	`AKIA...`	`"AKIA" filename:.env`

Stripe Key	`sk_live_...`	`"sk_live_" filename:.env`
Google API	`AIza...`	`"AIzaSy" filename:.env`
SSH Key	`BEGIN RSA PRIVATE KEY`	`"BEGIN RSA PRIVATE KEY" extension:pem`
GitHub Token	`ghp_...`	`"ghp_" site:github.com`
Slack Token	`xoxp-...`	`"xoxp-" OR "xoxb-" site:github.com`

Defensive Action Checklist

- [ ] Run `site:yourdomain.com filetype:env` monthly
- [ ] Search GitHub for your org name + secret patterns
- [ ] Enable secret scanning on all repositories
- [ ] Add pre-commit hooks to block secrets
- [ ] Review and restrict cloud storage ACLs
- [ ] Disable directory listing on web servers
- [ ] Implement proper robots.txt and noindex tags
- [ ] Rotate credentials on a schedule
- [ ] Monitor certificate transparency logs
- [ ] Set up Google Alerts for sensitive terms

Remember: All techniques in this document are for educational purposes only.

Only use on systems you own or have written permission to test.